

Job Hunter (Job + Gig Copilot)

An automated, end-to-end job search system: it fetches job postings from multiple sources, scores them against my resume using hybrid AI retrieval, writes tailored cover letters, fact-checks its own writing, and shows everything in a live dashboard — so I only spend time on applications actually worth sending.

1. Motive — why this exists

Manually searching multiple job boards, judging fit by eye, and writing a new cover letter from scratch for every application doesn't scale, and it's easy to miss good-fit roles simply because they're worded differently than what you'd search for. I wanted a system that:

- Pulls listings from several sources automatically instead of me checking each site by hand
- Judges *actual fit* against my resume using real retrieval techniques, not just keyword matching
- Drafts a genuinely tailored cover letter — grounded only in what my resume actually says, never invented
- Catches its own mistakes before I see them — both factual (did it lie?) and stylistic (does it sound like generic AI writing?)
- Shows me the whole pipeline's health, not just the end result, so I can trust — or debug — what it's doing

This is also a portfolio piece: it's meant to demonstrate real applied AI engineering (retrieval design, evaluation, multi-agent orchestration), not just "call an LLM and hope."

2. The solution, end to end

```
5 job sources (Adzuna, Himalayas, RemoteOK, Arbeitnow, Jobicy)
    ↓
  fetched postings → deduped by URL → Supabase (postings table)
    ↓
  resume.txt → chunked into sections → embedded (BGE) → Supabase (profile
table)
    ↓
  postings embedded (BGE) → coverage tracked (thin-description flag)
    ↓
  HYBRID RETRIEVAL:
  - pgvector cosine similarity (semantic)
  - BM25 keyword scoring (lexical)
  - Reciprocal Rank Fusion → combines both into one ranked list
```

```
- cross-encoder re-ranking on top 20 → final top 5
↓
every posting considered gets logged with a reason (match_log table) –
not just the winners, so "why didn't this match" is always answerable
↓
top 5 stored in matches table
↓
CrewAI drafting crew, 3 agents in sequence:
  Researcher (finds a real, specific company detail via web search)
    → Drafter (writes the letter, grounded only in resume facts)
    → Refiner (strips generic AI-sounding phrases, rewrites from scratch)
  ↓
EVALUATION (two independent checks):
  - Faithfulness: extract every factual claim → verify each one,
    one at a time, against the resume → % supported
  - Quality: scan for banned generic phrases → deterministic score
  ↓
stored back on matches table (draft + faithfulness/quality metadata)
↓
FastAPI backend exposes it all as clean HTTP endpoints
↓
dashboard (frontend) shows pipeline health, matches, drafts, and lets me
manually label matches as good/bad fit for a real precision@k metric
```

3. Why this tech, and not something else

Every choice below was made deliberately, with a real alternative considered and rejected — not just "the tutorial used it."

Retrieval: hybrid (vector + BM25 + RRF), not one method alone

- **Vector search alone** misses exact keyword matches — if a job says "LangChain" and my resume says "LangChain," pure semantic similarity can still rank it low if the surrounding context differs.
- **BM25 alone** misses meaning — it can't know "agentic systems" and "autonomous AI" mean roughly the same thing.
- **Combining both via Reciprocal Rank Fusion** catches what either method misses alone. This is the standard architecture for production search systems, not something invented for this project.
- **Honest limitation:** the BM25 step here re-ranks the same candidate pool vector search already surfaced, rather than searching the full posting set independently. Still useful (it catches lexical overlap cosine similarity can miss within that pool), but it's not textbook independent hybrid search — worth being precise about this if asked in an interview.

Cross-encoder re-ranking, on top of retrieval

Embedding comparison encodes the resume and each job *separately*, then compares vectors — fast, but the two texts never actually "see" each other. A cross-encoder reads both together and scores their relevance in context — far more accurate, but ~50x slower. So retrieval narrows 100+ postings down to 20 candidates cheaply, then the cross-encoder does the expensive, accurate pass only on those 20. This "retrieve then rerank" pattern is the standard architecture for production search.

A real bug we found and fixed: the cross-encoder's raw output (sigmoid of deeply negative logits, since the model was trained on short query/passage pairs, not resume/job-length text) produced meaningless scores clustered near zero. Fixed with min-max normalization per batch instead — ranking order stays identical, but the scores become interpretable (best in batch → 1.0, worst → 0.0).

BGE embeddings, not MiniLM

`BAAI/bge-small-en-v1.5` was chosen over the more common `all-MiniLM-L6-v2` because BGE was specifically trained for retrieval tasks and scores better on semantic similarity benchmarks. Same vector size (384-dim), drop-in replacement, better results. Runs fully locally — no API cost, no external dependency.

Supabase (Postgres + pgvector), not a dedicated vector DB

Dedicated vector databases (Pinecone, Weaviate, Chroma) are built specifically for embeddings and scale further. For a solo project, Supabase gives a normal relational database *and* vector search in one system — no syncing data between two separate stores, generous free tier. Would revisit this choice if scaling to millions of postings with heavy concurrent traffic.

CrewAI multi-agent drafting, not one prompt

Three agents, each with one narrow job, in sequence — not three prompts stapled together. The Researcher finds one genuinely interesting company detail (explicitly told to ignore financial/funding news, since that's what generic cover letters lead with). The Drafter writes under a hard constraint: every factual claim must trace back to an actual resume section. The Refiner rewrites any sentence containing a banned generic phrase from scratch, not just deletes the phrase and leaves a broken sentence.

A real bug we found and fixed: CrewAI was silently injecting a `cache_breakpoint` field into every message sent to the LLM, regardless of provider — a prompt-caching feature that only Anthropic's API understands. Groq rejected every request because of it. No documented environment variable stopped this; it happens at a level below where those variables take effect. Fixed with a targeted monkey-patch on

`crewai.llms.cache.mark_cache_breakpoint`.

Faithfulness eval — custom, RAGAS-inspired, not the literal RAGAS library

Two-step process: (1) extract every factual claim from a draft as a numbered list, (2) verify

each claim individually — one LLM call per claim — against the resume, answering only SUPPORTED or UNSUPPORTED. The score is $\frac{\text{supported}}{\text{total}}$. Verifying claim-by-claim, rather than asking "is this letter accurate overall," matters because LLMs asked for a holistic judgment tend to rubber-stamp "yes" even when specific claims are wrong.

Named honestly: this is *not* the actual RAGAS library — it's a hand-rolled equivalent. Calling it "RAGAS-inspired" is accurate; claiming literal RAGAS usage would not be.

Quality eval — separate from faithfulness, on purpose

A deterministic scan for banned generic phrases ("passionate," "leverage," "fast-paced," etc.), no LLM call. Faithfulness measures *truth*; quality measures *tone*. A letter can be 100% accurate and still read like a template — both need to pass independently, which is why they're two separate scores rather than one blended number.

Five job sources, chosen deliberately

- **Adzuna** — proper API with credentials, UK/US coverage
- **Himalayas** — no-auth public API, global remote roles
- **RemoteOK** — no-auth public JSON API, large remote listing volume
- **Arbeitsnow** — no-auth public API, EU-leaning remote roles
- **Jobicy** — no-auth public API, tag-based filtering

Deliberately excluded: LinkedIn. LinkedIn's User Agreement explicitly prohibits scraping or automated access; enforcement includes account bans and civil legal action, and the well-known hiQ Labs case only established that scraping public data isn't a *criminal* CFAA violation — it does nothing to protect against LinkedIn banning the account used to do it. Risking a real LinkedIn account (still needed for actual networking) for one more data source wasn't worth it. **Also excluded: Remote100K** — a real, well-regarded board, but browse-only with no public API; would require scraping.

Vite + React (originally) → Next.js (current) for the frontend

Started with Vite for the dashboard — no framework overhead needed since the frontend only reads from Supabase directly, no server-side rendering requirement. Rebuilt in Next.js partway through, primarily to pair with a dedicated FastAPI backend (trigger buttons, JD analysis, PDF generation) that a pure client-only Vite app couldn't cleanly support. **Note:** this transition is not yet fully reflected everywhere in project documentation — see Known Issues.

FastAPI backend, as the bridge

The frontend can't call Python pipeline scripts directly. FastAPI exposes clean HTTP endpoints that either query Supabase directly (fast reads) or trigger pipeline scripts as subprocesses (fetch/score/draft actions), deployed on Railway.

4. Repo structure

```
jobhunter/
├─ resume.txt           # personal resume – gitignored, never
committed
├─ schema.sql          # Supabase table definitions
|
├─ scrapers/           # Phase 2 – job source fetchers
|   ├─ adzuna.py        # Adzuna API fetcher
|   ├─ himalayas.py     # Himalayas API fetcher
|   ├─ remoteok.py      # RemoteOK API fetcher
|   ├─ arbeitnow.py     # Arbeitnow API fetcher
|   ├─ jobicy.py        # Jobicy API fetcher
|   ├─ store.py         # dedup + upsert postings into Supabase
|   ├─ notify.py        # plays a sound when a fetch run finishes
|   └─ run.py           # orchestrator – currently wires only 2 of
5 sources
|   └─ fetch_all_sources.py # orchestrator – wires all 5 sources +
sound
|
├─ pipeline/           # Phase 3 + 4 – embed, score, draft,
evaluate
|   ├─ embed.py         # BGE embeddings for resume + postings
|   └─ score.py         # hybrid retrieval: vector + BM25 + RRF +
cross-encoder
|   ├─ draft.py         # CrewAI 3-agent drafting crew
|   ├─ evaluate.py      # faithfulness + quality checks
|   ├─ run_phase3.py    # embed → score (standalone)
|   ├─ run_phase4.py    # draft → evaluate (standalone)
|   └─ pipeline.py      # merged embed → score → draft →
evaluate, one tracked run
|
├─ backend/            # Phase 5 – FastAPI bridge between frontend
and pipeline
|   ├─ Procfile         # Railway process definition
|   └─ railway.js       # Railway deployment config
|       └─ api/
|           ├─ main.py   # FastAPI app, CORS, route registration
|           └─ models.py # Pydantic request/response models
|               └─ routes/
|                   ├─ stats.py    # pipeline health metrics
|                   └─ matches.py  # paginated match list + detail +
status update
|                       ├─ analyze.py # paste-a-JD analysis endpoint
|                       └─ metrics.py  # precision@k, faithfulness/quality
distributions
```

```

|         | └─ pdf.py                # cover letter / fit report PDF
generation
|         | └─ trigger.py            # manually run fetch/score/draft from
the dashboard
|         └─ services/
|             └─ db.py                # Supabase client singleton
|             └─ jd_analyzer.py       # LLM chains behind /analyze-jd
|             └─ pdf_gen.py           # PDF rendering logic
|
└─ frontend/                         # Phase 5 – Next.js dashboard
    └─ app/
        └─ page.tsx                  # main dashboard page (only page
currently built)
        └─ layout.tsx                # root layout
        └─ components/
            └─ Card.tsx               # reusable stat/panel card
            └─ Nav.tsx                # top nav – links to 3 pages not yet
built
        └─ lib/
            └─ api.ts                 # typed fetch wrapper around the
FastAPI backend
|
└─ requirements.txt                  # Python dependencies (backend + pipeline)

```

5. What each file actually does

Scrapers

adzuna.py — Queries the Adzuna API across a list of AI/ML-relevant keywords and two markets (UK, US), normalizes results into the shared postings schema (title, company, description, url, pay_range, location, posted_at).

himalayas.py — Same normalization pattern, querying Himalayas' no-auth public API by keyword, paginated.

remoteok.py — RemoteOK returns its *entire* current listing set in one response (no server-side search) — this file pulls everything and filters client-side by keyword match against title/tags/description.

arbeitnow.py — Pages through Arbeitnow's API up to a page limit, filters for `remote: true` plus keyword match.

jobicy.py — Queries by broad industry tag (`data-science`, `dev`), then narrows further by keyword since the tag alone is too coarse.

`store.py` — Upserts normalized postings into Supabase, deduped by a unique constraint on `url` — running fetchers repeatedly never creates duplicate rows.

`notify.py` — Plays a short macOS system sound (`afplay`) when a run finishes; falls back to a terminal bell on other platforms so it never crashes the script.

`run.py` — The fetch orchestrator currently wired to the dashboard's "Run fetch" trigger.
Currently only calls Adzuna and Himalayas — see Known Issues.

`fetch_all_sources.py` — The orchestrator that calls all five sources, stores results, prints a full summary, and plays the completion sound. Fully built but **not currently referenced by anything** — see Known Issues.

Pipeline

`embed.py` — Loads BGE once (cached), embeds resume text and unembedded postings, tracks `description_word_count` per posting to power a coverage metric (what % of postings had enough text to embed meaningfully). Also chunks the resume into labeled sections (experience, education, skills, etc.) for targeted retrieval by the drafting agents.

`score.py` — The core retrieval engine:

- `_vector_retrieve`: pgvector cosine similarity via Supabase RPC
- `_bm25_score`: keyword scoring against the same candidate pool
- `_rrf_combine`: Reciprocal Rank Fusion merges both rankings
- `_cross_encoder_rerank`: re-ranks the top 20 with min-max normalized scores
- `_log_considered_postings`: writes a row to `match_log` for every posting considered, with a reason (`thin_description`), `rrf_ranked_below_top20`, `cross_encoder_ranked_below_top5`, or `selected`) — this is what powers "why didn't this match" on the dashboard
- `_store_matches`: upserts the final top 5 into `matches`

`draft.py` — The CrewAI drafting crew: Researcher (Tavily web search) → Drafter (writes under strict resume-only constraints) → Refiner (removes banned generic phrases, rewrites affected sentences from scratch). Includes the `cache_breakpoint` monkey-patch fix for the CrewAI/Groq bug.

`evaluate.py` — Two independent scoring functions:

- `score_faithfulness`: extract claims → verify each individually → % supported
- `score_quality`: scan for banned phrases, deterministic, no LLM call

`run_phase3.py` / `run_phase4.py` — Standalone entry points for embed+score and draft+evaluate respectively, useful for debugging one stage in isolation.

`pipeline.py` — The merged, fully-tracked run: embed → score → draft → evaluate in one command, with timestamped logging, elapsed-time tracking, and deliberate pauses

between LLM calls to stay under Groq's rate limits.

Backend (FastAPI)

`main.py` — App setup, CORS middleware, registers all 6 route groups under `/api`.

`routes/stats.py` — Pipeline health: posting/embedding/match/draft counts, coverage %, score distribution buckets, source breakdown, last run timestamp.

`routes/matches.py` — Paginated match listing (with posting details joined in), single match detail, and a PATCH endpoint to set manual status/human_label for building a real precision@k dataset.

`routes/analyze.py` — Paste any job description, get back a role summary, a resume fit analysis, and generated interview questions — stored in a `jd_analyses` table for later reference.

`routes/metrics.py` — Computes precision@5/@10 from manually-labeled matches, plus faithfulness and quality score distributions (mean/min/max) across all evaluated drafts.

`routes/pdf.py` — Generates downloadable PDFs (cover letter, fit report) on demand.

`routes/trigger.py` — Runs fetch/score/draft pipeline scripts as subprocesses from a dashboard button click, so the terminal never has to be opened for routine runs.

`services/db.py` — Single shared Supabase client instance.

`services/jd_analyzer.py` — The LLM chains behind `/analyze-jd`.

`services/pdf_gen.py` — PDF rendering logic behind `/pdf`.

Frontend (Next.js)

`app/page.tsx` — The dashboard: pipeline stat cards (postings, embedding coverage, matches, draft success rate), a score distribution chart, a postings-by-source chart, and manual trigger buttons for fetch/score/draft.

`components/Card.tsx` — Shared visual container for stat blocks.

`components/Nav.tsx` — Top navigation, links to Dashboard/Matches/Analyze JD/Metrics.

`lib/api.ts` — Typed `get`/`post`/`patch` wrapper around the FastAPI backend, reads the backend URL from `NEXT_PUBLIC_API_URL`.

6. Phase progress

- **Phase 1 — Schema & infrastructure.** Supabase tables deployed (`postings`, `profile`, `matches`, `applications`), plus `match_log`, `jd_analyses` added later). 
Caveat: `schema.sql` in the repo is currently empty — the live schema exists in Supabase but isn't committed anywhere as source of truth. See Known Issues.

- **Phase 2 — Fetchers.** Five source fetchers built (Adzuna, Himalayas, RemoteOK, Arbeitnow, Jobby), deduped storage via unique URL constraint. ✅ built, ⚠️ **only 2 of 5 currently wired to the actual run path** — see Known Issues.
 - **Phase 3 — Embed + Score.** Resume and postings embedded via BGE, hybrid scoring (vector + BM25 + RRF + cross-encoder) working, top 5 stored per run, every candidate logged with a match/no-match reason. ✅ Complete and verified working.
 - **Phase 4 — Draft + Evaluate.** CrewAI 3-agent drafting crew, faithfulness eval (claim-by-claim verification), quality eval (banned-phrase scan). ✅ Complete. ⚠️ **Token-overflow truncation fix for long job descriptions was designed but never actually landed in the deployed (`draft.py`)** — see Known Issues.
 - **Phase 5 — Dashboard.** FastAPI backend (6 route groups) + Next.js frontend. ✅ Backend fully built. ⚠️ Frontend has only 1 of 4 planned pages built (Dashboard) — Matches, Analyze JD, and Metrics pages are linked in the nav but don't exist yet, so those links currently 404.
 - **Phase 6 — Deployment.** Railway config exists for the backend (`Procfile`), (`railway.js`). ➡️ SOON Vercel deployment for frontend planned but not yet confirmed live. Private-access gating (auth) not yet implemented.
-

7. Critique received so far, and status

An AI/ML specialist and a recruiter reviewed the project. Summary of what they said, and what's been done about each point:

Critique	Status
No proof retrieval quality beats a naive baseline	⚠️ Metrics endpoint built, but the precision@k formula has a bug (see Known Issues) — not yet a trustworthy number
No demo video / screenshots in README	➡️ SOON Not yet done
"Hybrid" retrieval isn't fully independent (BM25 reranks vector's pool, not the full set)	Acknowledged, documented honestly above — not a code fix, a framing accuracy issue
No tests for math-heavy functions (<code>_rrf_combine</code> , <code>_normalize</code> , <code>_chunk_resume</code>)	➡️ SOON Not yet done — no test files exist in the repo
Banned-phrase list duplicated across files	➡️ SOON Not yet centralized
<code>requirements.txt</code> has a duplicate <code>langchain-groq</code> line	➡️ SOON Not yet fixed
Cross-encoder sigmoid producing meaningless scores	✅ Fixed — min-max normalization
CrewAI <code>cache_breakpoint</code> bug blocking all Groq calls	✅ Fixed — monkey-patch applied

8. Known issues (found during a full repo audit)

These were found by cloning the actual public repo and checking it file by file against what documentation and prior planning claimed — not just going off memory of what was intended.

- Trigger button for "fetch" points at the wrong path.** `backend/api/routes/trigger.py` looks for `pipeline/run.py`, but the real fetcher script lives at `scrapers/run.py`. Clicking "Run fetch" on the dashboard currently fails.
- Only 2 of 5 fetchers are wired into the active run path.** `scrapers/run.py` (what the trigger button calls, once the path above is fixed) only calls Adzuna and Himalayas. `fetch_all_sources.py` calls all five but is currently orphaned code.
- The Groq token-truncation fix isn't in `draft.py`.** Designed to solve the exact rate-limit/token-overflow problem hit during development, but the function was never added to the deployed file — the underlying problem is likely still there.
- `pipeline_runs` table is read but never written.** `stats.py` queries it for a "last run" timestamp; nothing anywhere inserts into it, so that field will always show as empty.

5. `schema.sql` is empty. No committed source of truth for the database structure, despite it being referenced throughout the docs.
 6. 3 of 4 frontend nav links are dead. `/matches`, `/analyze`, and `/metrics` are linked in `Nav.tsx` but don't exist under `frontend/app/` yet — currently 404.
 7. `precision@k` formula inflates the score. Currently computed as $\frac{\text{good_labels_in_top_k}}{\text{labeled_items_in_top_k}}$ instead of $\frac{\text{good_labels_in_top_k}}{k}$ — a mostly-unlabeled top-5 can show a misleadingly perfect precision@5.
 8. CORS wildcard likely non-functional. `allow_origins` includes `"https://*.vercel.app"`, but FastAPI's `CORSMiddleware` does exact string matching, not glob matching — needs `allow_origin_regex` instead for this to actually work in production.
 9. `ragas` is a dead dependency. Listed in `requirements.txt`, never imported anywhere — the faithfulness eval is a custom implementation, not literal RAGAS, so this dependency is both unused and slightly misleading to anyone reading `requirements.txt` at face value.
 10. `langchain-groq` is listed twice in `requirements.txt`.
-

9. What's planned next

Roughly in priority order:

1. Fix `trigger.py`'s path bug, and repoint it at `fetch_all_sources.py` so all 5 sources actually run from the dashboard, not just 2.
2. Re-apply the token truncation fix to `draft.py` — actively blocking reliable drafting runs right now.
3. Fix the CORS `allow_origin_regex` before depending on the deployed Vercel + Railway setup working end to end.
4. Commit a real `schema.sql`, exported from the live Supabase schema.
5. Build the 3 missing frontend pages (`/matches`, `/analyze`, `/metrics`) — the backend endpoints for all three already exist and work.
6. Fix the `precision@k` formula to divide by `k`, not by labeled count.
7. Add `pipeline_runs` inserts so "last run" actually populates on the dashboard.
8. Record a demo video/GIF for the README — still the single highest dashboard-visibility fix, unrelated to any code change.
9. Run the manual `precision@k` eval properly — label 20-30 matches, compute vector-only vs. full hybrid pipeline precision, add the real number (not the currently-buggy one) to this README.
10. Lower priority: centralize the banned-phrases list, add pytest coverage for `_rrf_combine`/`_normalize`/`_chunk_resume`, clean up `requirements.txt`, add

email notifications on strong new matches, add the tailored-resume bullet-reordering feature, add a minimum score floor so weak batches don't force through 5 mediocre matches.

10. Setup

Backend / pipeline:

```
bash
cd backend # or wherever your .env lives
```

Create a `.env` with:

```
SUPABASE_URL=https://YOUR_PROJECT_ID.supabase.co
SUPABASE_KEY=your_supabase_key
GROQ_API_KEY=your_groq_key
TAVILY_API_KEY=your_tavily_key
ADZUNA_APP_ID=your_adzuna_app_id
ADZUNA_APP_KEY=your_adzuna_app_key
```

```
bash
pip install -r requirements.txt
python scrapers/fetch_all_sources.py # bring in fresh postings (all 5 sources)
python pipeline/pipeline.py         # embed → score → draft → evaluate
```

FastAPI backend (local):

```
bash
cd backend
uvicorn api.main:app --reload --port 8000
```

Frontend:

```
bash
cd frontend
npm install
# create .env.local with NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev
```